# Complete Terraform Guide: From Start to End

## Table of Contents


1. Introduction to Infrastructure as Code
2. Why Terraform?
3. Terraform Installation
4. Terraform Basic Concepts
5. Terraform Workflow
6. Azure Authentication Methods
7. Terraform Resources and Documentation
8. Terraform State File
9. Terraform Providers
10. Terraform Variables
11. Conditional Expressions
12. Built-in Functions
13. Terraform Modules
14. Debugging and Troubleshooting
15. Best Practices


---

## Introduction to Infrastructure as Code

### What is Infrastructure as Code (IaC)?

Infrastructure as Code is the practice of defining and managing cloud infrastructure using code instead of manually configuring it through cloud consoles.

**Problem with Manual Infrastructure Management:**

- Slow and time-consuming
- Prone to human errors
- Inconsistent across environments
- Impossible to scale reliably
- No version control or audit trail

**Example Scenario:**
Imagine setting up a cloud environment manually by clicking through AWS/Azure console. You spend hours creating EC2 instances, networking, security groups, and storage. One week later, your colleague needs the exact same environment for a different project. You try to replicate the steps but:

- You don't remember every detail
- You make mistakes
- Misconfigure security groups
- Forget to attach storage
- Overprovision resources

This leads to inconsistency and wasted time debugging.

## How IaC Solves These Problems

- **Consistency**: Every deployment produces identical infrastructure
- **Automation**: Resources created in minutes instead of hours
- **Version Control**: Track changes, rollback to known good states
- **Collaboration**: Multiple team members can work on infrastructure
- **Scalability**: Deploy hundreds of resources with one command
- **Cost Efficiency**: Delete resources when not in use, recreate when needed

## Imperative vs Declarative IaC

**Imperative Approach:**

- Step-by-step instructions
- You explicitly define each action
- Example: Azure CLI commands

```
az group create --name my-rg --location eastus
```

- If the resource already exists, it will fail
- You must handle errors manually

**Declarative Approach:**

- Define the end goal (desired state)
- System figures out how to achieve it
- Example: Terraform configuration

```
resource "azurerm_resource_group" "example" {
  name     = "my-rg"
  location = "eastus"
}
```

- If resource exists with correct settings → do nothing
- If resource exists with incorrect settings → update it

- If resource doesn't exist → create it

---

# Why Terraform?

## The Multi-Cloud Challenge

Imagine your organization uses:

- AWS for development
- Azure for data team
- GCP for security

Each cloud has its own IaC tool:

- **AWS CloudFormation** → Only works with AWS
- **ARM Templates** → Only works with Azure
- **OpenStack Heat** → Only works with OpenStack

## Terraform's Unique Advantages

1. **Cloud Agnostic (No Vendor Lock-in)**
   - Deploy across AWS, Azure, GCP simultaneously
   - Manage on-premises infrastructure
   - Even work with SaaS providers (Datadog, GitHub, etc.)
2. **Declarative Language (HCL)**
   - Simple and readable
   - Easier to maintain than JSON/YAML templates
3. **State Management**
   - Tracks all resources created
   - Single source of truth
   - Enables collaboration
4. **Modular and Reusable**
   - Write once, use anywhere
   - Share modules with teams
5. **Extensible Plugin Architecture**
   - 1000+ providers available
   - Community-supported integrations

---

# Terraform Installation

## Installation Methods by OS

**Windows (using Chocolatey):**

```
choco install terraform
```

**macOS (using Homebrew):**

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

**Linux:**

```
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | sudo tee
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform
```

## Verify Installation

```
terraform --version
```

## VS Code Extensions (Essential)

1. **HashiCorp Terraform** - Syntax highlighting and auto-completion
2. **HashiCorp HCL** - HCL language support

---

# Terraform Basic Concepts

## File Structure and Naming

- Terraform files use `.tf` extension
- Main configuration file is typically `main.tf`
- Files can be split for better organization:
  - `main.tf` → Core resource definitions
  - `variables.tf` → Input variable declarations

- `outputs.tf` → Output value definitions
- `providers.tf` → Provider configurations
- `terraform.tfvars` → Variable values

## Three Key Blocks in Terraform

1. **Provider Block** - Defines which cloud/service to interact with

```
provider "azurerm" {
  features {}
}
```

2. **Resource Block** - Defines what infrastructure to create

```
resource "azurerm_resource_group" "example" {
  name     = "terraform-example-rg"
  location = "East US"
}
```

3. **Output Block** - Returns values after deployment

```
output "resource_group_name" {
  value = azurerm_resource_group.example.name
}
```

---

# Terraform Workflow

## The Core Workflow

```
Write → Initialize → Plan → Apply → Modify → Destroy
```

## Detailed Steps

### 1. Write Configuration

- Create `.tf` files with desired infrastructure

### 2. Initialize ( `terraform init` )

```
terraform init
```

- Downloads required provider plugins
- Initializes backend for state storage
- Creates `.terraform` directory and lock file

**3. Plan (`terraform plan`)**

```
terraform plan
```

- Previews changes before applying
- Shows what will be added, changed, or destroyed
- No actual infrastructure changes

**4. Apply (`terraform apply`)**

```
terraform apply
```

- Creates/updates infrastructure
- Asks for confirmation (use `-auto-approve` to skip)
- Updates state file

**5. Modify**

- Update `.tf` files
- Run `plan` and `apply` again

**6. Destroy (`terraform destroy`)**

```
terraform destroy
```

- Deletes all managed resources
- Asks for confirmation

---

# Azure Authentication Methods

## Method 1: Azure CLI Login (Simple but Not Recommended for Production)

```
# Login to Azure
az login --use-device-code

# Verify subscription
az account show

# Get subscription ID
az account show --query id -o tsv
```

In `main.tf`:

```
provider "azurerm" {
  features {}
  subscription_id = "your-subscription-id"
}
```

## Method 2: Service Principal (Recommended)

**What is a Service Principal?**

- An identity for applications/services to access Azure
- Like a user account but for automation
- More secure than using personal credentials

**Create Service Principal:**

```
az ad sp create-for-rbac --name "terraform" --role Contributor --scopes
/subscriptions/your-subscription-id
```

**Output (example):**

```
{
  "appId": "5e2cbe25-60ed-49b0-9750-8ef101c4ea1a",
  "displayName": "terraform",
  "password": "K5l8Q~bFdFlzuGKVpJ8B73mOp7r6BvlY2JON2cHg",
  "tenant": "227e0b5a-6643-4742-bb94-e90cad4cd284"
}
```

**Set Environment Variables (Windows PowerShell):**

```
[System.Environment]::SetEnvironmentVariable("ARM_CLIENT_ID", "5e2cbe25-60ed-49b0-9750-8ef101c4ea1a", "User")
[System.Environment]::SetEnvironmentVariable("ARM_CLIENT_SECRET", "K5l8Q~bFdFlzuGKVpJ8B73mOp7r6BvlY2JON2cHg", "User")
[System.Environment]::SetEnvironmentVariable("ARM_SUBSCRIPTION_ID", "1ff9300b-26c8-4219-93dc-99a8261c64ef", "User")
[System.Environment]::SetEnvironmentVariable("ARM_TENANT_ID", "227e0b5a-6643-4742-bb94-e90cad4cd284", "User")
```

**Set Environment Variables (Linux/macOS):**

```
export ARM_CLIENT_ID="5e2cbe25-60ed-49b0-9750-8ef101c4ea1a"
export ARM_CLIENT_SECRET="K5l8Q~bFdFlzuGKVpJ8B73mOp7r6BvlY2JON2cHg"
export ARM_SUBSCRIPTION_ID="1ff9300b-26c8-4219-93dc-99a8261c64ef"
export ARM_TENANT_ID="227e0b5a-6643-4742-bb94-e90cad4cd284"
```

**Make Permanent (Linux/macOS):**
Add to `~/.bashrc` or `~/.zshrc`:

```
echo 'export ARM_CLIENT_ID="5e2cbe25-60ed-49b0-9750-8ef101c4ea1a"' >> ~/.bashrc
echo 'export ARM_CLIENT_SECRET="K5l8Q~bFdFlzuGKVpJ8B73mOp7r6BvlY2JON2cHg"' >> ~/.bashrc
source ~/.bashrc
```

**Security Best Practices:**

- Only assign required permissions (least privilege)
- Store credentials securely (Key Vault, GitHub Secrets)
- Rotate credentials regularly
- Use federated identity when possible

---

# Terraform Resources and Documentation

## Understanding Resources

A Terraform resource represents a specific piece of infrastructure managed by Terraform.

**Resource Structure:**

```
resource "<provider_resource_type>" "<logical_name>" {
  # Configuration parameters
}
```

**Example:**

```
resource "azurerm_virtual_machine" "example_vm" {
  name                = "my-vm"
  resource_group_name = azurerm_resource_group.example_rg.name
  location            = azurerm_resource_group.example_rg.location
  size                = "Standard_B1s"
}
```

## Finding Documentation

**Terraform Registry:** https://registry.terraform.io/

**Azure Provider Documentation:** https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs

**Steps to Find Resource Code:**

1. Go to Terraform Registry
2. Browse Providers → Azure
3. Search for resource (e.g., "storage account")
4. Copy example code
5. Customize as needed

## Common Azure Resources

**Resource Group:**

```
resource "azurerm_resource_group" "example" {
  name     = "my-resource-group"
  location = "East US"
}
```

**Virtual Network:**

```
resource "azurerm_virtual_network" "example" {
  name                = "my-vnet"
  location            = azurerm_resource_group.example.location
  resource_group_name = azurerm_resource_group.example.name
  address_space       = ["10.0.0.0/16"]
}
```

**Subnet:**

```
resource "azurerm_subnet" "example" {
  name                 = "my-subnet"
  resource_group_name  = azurerm_resource_group.example.name
  virtual_network_name = azurerm_virtual_network.example.name
  address_prefixes     = ["10.0.1.0/24"]
}
```

**Network Interface:**

```
resource "azurerm_network_interface" "example" {
  name                = "my-nic"
  location            = azurerm_resource_group.example.location
  resource_group_name = azurerm_resource_group.example.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.example.id
    private_ip_address_allocation = "Dynamic"
  }
}
```

**Public IP:**

```
resource "azurerm_public_ip" "example" {
  name                = "my-public-ip"
  location            = azurerm_resource_group.example.location
  resource_group_name = azurerm_resource_group.example.name
  allocation_method   = "Static"
}
```

**Virtual Machine:**

```
resource "azurerm_virtual_machine" "example" {
  name                  = "my-vm"
  location              = azurerm_resource_group.example.location
  resource_group_name   = azurerm_resource_group.example.name
  network_interface_ids = [azurerm_network_interface.example.id]
  vm_size               = "Standard_B1s"

  os_profile {
    computer_name  = "myvm"
    admin_username = "adminuser"
    admin_password = "P@ssw0rd!"
  }

  os_profile_linux_config {
    disable_password_authentication = false
  }

  storage_image_reference {
    publisher = "Canonical"
    offer     = "UbuntuServer"
    sku       = "18.04-LTS"
    version   = "latest"
  }

  storage_os_disk {
    name              = "my-os-disk"
    caching           = "ReadWrite"
    create_option     = "FromImage"
    managed_disk_type = "Standard_LRS"
  }
}
```

**Storage Account:**

```
resource "azurerm_storage_account" "example" {
  name                     = "mystorageaccount123"
  resource_group_name      = azurerm_resource_group.example.name
  location                 = azurerm_resource_group.example.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
}
```

# Terraform State File

## What is Terraform State?

Terraform maintains a state file ( `terraform.tfstate` ) that:

- Tracks all deployed infrastructure
- Maintains resource dependencies
- Maps configuration to real-world resources
- Stores metadata about resources

## How State Works

When you run `terraform apply` :

1. Terraform reads the current state file
2. Reads the configuration ( `.tf` files)
3. Compares desired vs actual state
4. Creates plan for changes
5. Applies changes and updates state

## What's in the State File?

The state file contains:

- Resource IDs
- Resource attributes (name, location, etc.)
- Dependencies
- Output values
- Sensitive information (passwords, keys)

## State File Security

**Sensitive Information in State:**

- Cloud provider credentials
- Database passwords
- Private IP addresses
- Access keys

**Best Practices:**

1. **Never commit state to Git** - Add to `.gitignore`

```
terraform.tfstate
terraform.tfstate.backup
.terraform/
```

2. **Use Remote State Storage** - Store state centrally
   - AWS S3 with DynamoDB locking
   - Azure Blob Storage
   - Terraform Cloud
3. **Enable Encryption** - Encrypt state at rest
4. **Use Role-Based Access Control** - Restrict access

## Remote State Example (Azure)

```
terraform {
  backend "azurerm" {
    resource_group_name  = "terraform-state-rg"
    storage_account_name = "terraformstate123"
    container_name       = "terraform-state"
    key                  = "prod.terraform.tfstate"
  }
}
```

## State Commands

**List resources in state:**

```
terraform state list
```

**Show specific resource:**

```
terraform state show azurerm_resource_group.example
```

**Move resource in state:**

```
terraform state mv azurerm_resource_group.old azurerm_resource_group.new
```

**Remove resource from state (but keep in cloud):**

```
terraform state rm azurerm_resource_group.example
```

# Terraform Providers

## What are Providers?

Providers are plugins that allow Terraform to interact with:

- Cloud providers (AWS, Azure, GCP)
- SaaS platforms (Datadog, GitHub)
- Infrastructure tools (Kubernetes)
- On-premises systems

## Provider Categories

**Official (HashiCorp Maintained):**

- AWS, Azure, GCP, Kubernetes
- Fully tested and supported

**Partner:**

- Maintained by third-party companies
- Alibaba Cloud, Oracle Cloud

**Community:**

- Maintained by community developers

## Provider Configuration

**Azure:**

```
provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

**AWS:**

```
provider "aws" {
  region = "us-east-1"
}
```

**Google Cloud:**

```
provider "google" {
  project = "my-project"
  region  = "us-central1"
}
```

## Multi-Cloud Configuration

Create resources in multiple clouds with one configuration:

```
# Provider for AWS
provider "aws" {
  region = "us-east-1"
}

# Provider for Azure
provider "azurerm" {
  features {}
}

# AWS S3 Bucket
resource "aws_s3_bucket" "example" {
  bucket = "my-multicloud-bucket"
}

# Azure Resource Group
resource "azurerm_resource_group" "example" {
  name     = "multicloud-rg"
  location = "East US"
}
```

## Multi-Region Configuration

Use aliases to configure multiple regions:

```
# East US provider
provider "azurerm" {
  alias    = "eastus"
  features {}
}

# West US provider
provider "azurerm" {
  alias    = "westus"
  features {}
}

# Resource group in East US
resource "azurerm_resource_group" "east" {
  provider = azurerm.eastus
  name     = "rg-eastus"
  location = "East US"
}

# Resource group in West US
resource "azurerm_resource_group" "west" {
  provider = azurerm.westus
  name     = "rg-westus"
  location = "West US"
}
```

---

# Terraform Variables

## The Problem with Hardcoded Values

When configurations are hardcoded:

- Not reusable across environments
- Difficult to maintain
- Prone to errors

## Variable Types

**Input Variables** - Pass dynamic values into Terraform
**Output Variables** - Retrieve values from deployed resources
**Local Values** - Intermediate calculations

## Defining Input Variables (`variables.tf`)

```
variable "resource_group_name" {
  description = "Name of the resource group"
  type        = string
  default     = "my-rg"
}

variable "location" {
  description = "Azure region"
  type        = string
  default     = "East US"
}

variable "vm_size" {
  description = "Size of the virtual machine"
  type        = string
  default     = "Standard_B1s"
}

variable "tags" {
  description = "Resource tags"
  type        = map(string)
  default = {
    Environment = "Development"
    Project     = "Terraform"
  }
}

variable "admin_password" {
  description = "Admin password"
  type        = string
  sensitive   = true  # Won't be shown in output
}
```

## Using Variables (`main.tf`)

```
resource "azurerm_resource_group" "example" {
  name     = var.resource_group_name
  location = var.location
  tags     = var.tags
}

resource "azurerm_virtual_machine" "example" {
  name     = var.vm_name
  location = var.location
  vm_size  = var.vm_size
  # ... other configuration
}
```

## Defining Output Variables (`outputs.tf`)

```
output "resource_group_name" {
  description = "The name of the resource group"
  value       = azurerm_resource_group.example.name
}

output "vm_private_ip" {
  description = "Private IP address of the VM"
  value       = azurerm_network_interface.example.private_ip_address
}

output "vm_public_ip" {
  description = "Public IP address of the VM"
  value       = azurerm_public_ip.example.ip_address
}
```

## Variable Precedence (Highest to Lowest)

1. Command line `-var` or `-var-file`
2. `terraform.tfvars` or `terraform.tfvars.json`
3. `*.auto.tfvars` files
4. Environment variables (`TF_VAR_variable_name`)
5. Variable defaults

## Setting Variable Values

**Command line:**

```
terraform apply -var="resource_group_name=my-prod-rg" -
var="vm_size=Standard_D4s_v3"
```

**Environment variables:**

```
export TF_VAR_resource_group_name="my-prod-rg"
export TF_VAR_admin_password="secure123"
terraform apply
```

**Variable files:**

`terraform.tfvars` (default):

```
resource_group_name = "my-dev-rg"
vm_size = "Standard_B1s"
environment = "dev"
```

`prod.tfvars` (environment-specific):

```
resource_group_name = "my-prod-rg"
vm_size = "Standard_D4s_v3"
environment = "prod"
```

```
terraform apply -var-file="prod.tfvars"
```

## Variable Type Validation

```
variable "vm_size" {
  description = "VM size"
  type        = string

  validation {
    condition     = contains(["Standard_B1s", "Standard_D2s_v3",
"Standard_D4s_v3"], var.vm_size)
    error_message = "VM size must be one of: Standard_B1s, Standard_D2s_v3,
Standard_D4s_v3."
  }
}
```

## Local Values

Use locals for intermediate calculations:

```
locals {
  common_tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
    Project     = var.project_name
  }

  resource_name_prefix = "${var.project_name}-${var.environment}"
}

resource "azurerm_resource_group" "example" {
  name     = "${local.resource_name_prefix}-rg"
  location = var.location
  tags     = local.common_tags
}
```

---

# Conditional Expressions

## What are Conditional Expressions?

Also known as ternary operators, they provide a compact way to write if-else logic in a single line.

**Syntax:**

```
condition ? true_value : false_value
```

## Basic Example

```
# Check if age is >= 18
age = 20
status = age >= 18 ? "adult" : "minor"
# status = "adult"
```

## Use Case 1: Conditional Resource Creation

Create public IP only in development, not in production:

```
resource "azurerm_public_ip" "example" {
  count = var.environment == "production" ? 0 : 1

  name                = "my-public-ip"
  location            = var.location
  resource_group_name = azurerm_resource_group.example.name
  allocation_method   = "Static"
}
```

## Use Case 2: Conditional Resource Configuration

Choose different VM sizes based on environment:

```
resource "azurerm_virtual_machine" "example" {
  name     = var.vm_name
  location = var.location
  vm_size  = var.environment == "production" ? "Standard_D4s_v3" : "Standard_B1s"
  # ... other configuration
}
```

## Use Case 3: Conditional Disk Size

```
resource "azurerm_managed_disk" "example" {
  name                 = "my-disk"
  location             = var.location
  resource_group_name  = azurerm_resource_group.example.name
  storage_account_type = "Standard_LRS"
  create_option        = "Empty"
  disk_size_gb         = var.environment == "production" ? 512 : 128
}
```

## Use Case 4: Conditional Tag

```
locals {
  environment_tag = var.environment == "production" ? "prod" : "dev"

  tags = {
    Environment = local.environment_tag
    ManagedBy   = "Terraform"
  }
}
```

# Built-in Functions

## What are Built-in Functions?

Terraform provides built-in functions to:

- Manipulate strings
- Perform mathematical calculations
- Work with collections (lists, maps)
- Handle encoding and decoding

## Testing Functions with `terraform console`

```
terraform console
```

## String Functions

**upper() / lower():**

```
> upper("terraform")
"TERRAFORM"

> lower("Terraform123")
"terraform123"
```

**replace():**

```
> replace("terraform-lab", "lab", "production")
"terraform-production"
```

**format():**

```
> format("%s-%s-rg", "terraform", "lab")
"terraform-lab-rg"
```

**join():**

```
> join("-", ["dev", "server", "01"])
"dev-server-01"
```

**split():**

```
> split("-", "dev-server-01")
["dev", "server", "01"]
```

## Numeric Functions

**max() / min():**

```
> max(10, 20, 5)
20

> min(50, 30, 100)
30
```

**floor() / ceil():**

```
> floor(9.7)
9

> ceil(9.2)
10
```

**pow():**

```
> pow(2, 3)
8
```

## Collection Functions

**length():**

```
> length(["vm1", "vm2", "vm3"])
3
```

**element():**

```
> element(["small", "medium", "large"], 1)
"medium"
```

**lookup():**

```
> lookup({name="John", age=30}, "name", "Unknown")
"John"
```

## Practical Examples

**Dynamic Resource Naming:**

```
# Resource group name: terraform-lab-rg
resource "azurerm_resource_group" "example" {
  name = format("%s-%s-rg", lower(var.project_name), var.environment)
  location = var.location
}

# VM name: terraform-lab-vm
resource "azurerm_virtual_machine" "example" {
  name = format("%s-%s-vm", lower(var.project_name), var.environment)
  # ... other configuration
}

# Subnet name: terraform-lab-subnet
resource "azurerm_subnet" "example" {
  name = join("-", [lower(var.project_name), var.environment, "subnet"])
  # ... other configuration
}
```

**Dynamic Disk Size:**

```
variable "base_disk_size" {
  type    = number
  default = 100
}

resource "azurerm_managed_disk" "example" {
  disk_size_gb = var.environment == "production" ? var.base_disk_size * 2 :
var.base_disk_size
  # ... other configuration
}
```

---

# Terraform Modules

## What is a Module?

A module is a container for multiple resources that are used together. It's a reusable, encapsulated piece of infrastructure.

**Benefits:**

- Reduce code duplication
- Improve maintainability
- Enforce best practices
- Enable team collaboration
- Faster deployment

## Module Structure

```
terraform-modules/
└── vnet-module/
    ├── main.tf       # Resource definitions
    ├── variables.tf  # Input variables
    └── outputs.tf    # Output values
```

## Creating a VNet Module

`vnet-module/main.tf`:

```
resource "azurerm_virtual_network" "vnet" {
  name                = var.vnet_name
  location            = var.location
  resource_group_name = var.resource_group_name
  address_space       = var.address_space
}

resource "azurerm_subnet" "subnet" {
  name                 = var.subnet_name
  resource_group_name  = var.resource_group_name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = var.subnet_prefixes
}
```

`vnet-module/variables.tf`:

```
variable "vnet_name" {
  description = "Name of the Virtual Network"
  type        = string
}

variable "location" {
  description = "Azure region"
  type        = string
}

variable "resource_group_name" {
  description = "Resource Group name"
  type        = string
}

variable "address_space" {
  description = "Address space for VNet"
  type        = list(string)
}

variable "subnet_name" {
  description = "Subnet name"
  type        = string
}

variable "subnet_prefixes" {
  description = "Subnet address prefixes"
  type        = list(string)
}
```

**`vnet-module/outputs.tf`:**

```
output "vnet_id" {
  description = "The ID of the Virtual Network"
  value       = azurerm_virtual_network.vnet.id
}

output "subnet_id" {
  description = "The ID of the subnet"
  value       = azurerm_subnet.subnet.id
}
```

## Using a Module

**Root `main.tf`:**

```
provider "azurerm" {
  features {}
}

# Create resource group
resource "azurerm_resource_group" "example" {
  name     = "terraform-lab-rg"
  location = "East US"
}

# Use VNet module
module "network" {
  source = "./terraform-modules/vnet-module"

  vnet_name           = "example-vnet"
  location            = azurerm_resource_group.example.location
  resource_group_name = azurerm_resource_group.example.name
  address_space       = ["10.0.0.0/16"]
  subnet_name         = "example-subnet"
  subnet_prefixes     = ["10.0.1.0/24"]
}

# Create VM using subnet from module
resource "azurerm_network_interface" "example_nic" {
  name                = "example-nic"
  location            = azurerm_resource_group.example.location
  resource_group_name = azurerm_resource_group.example.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = module.network.subnet_id
    private_ip_address_allocation = "Dynamic"
  }
}

resource "azurerm_virtual_machine" "example_vm" {
  name                  = "example-vm"
  location              = azurerm_resource_group.example.location
  resource_group_name   = azurerm_resource_group.example.name
  network_interface_ids = [azurerm_network_interface.example_nic.id]
  vm_size               = "Standard_B1s"
```

```
  os_profile {
    computer_name  = "example-vm"
    admin_username = "adminuser"
    admin_password = "P@ssw0rd!"
  }

  os_profile_linux_config {
    disable_password_authentication = false
  }

  storage_image_reference {
    publisher = "Canonical"
    offer     = "UbuntuServer"
    sku       = "18.04-LTS"
    version   = "latest"
  }

  storage_os_disk {
    name              = "example-os-disk"
    caching           = "ReadWrite"
    create_option     = "FromImage"
    managed_disk_type = "Standard_LRS"
  }
}
```

## Module Sources

**Local Path:**

```
module "network" {
  source = "./terraform-modules/vnet-module"
}
```

**GitHub:**

```
module "network" {
  source = "github.com/your-org/terraform-modules//vnet-module"
}
```

**Terraform Registry:**

```
module "vnet" {
  source  = "Azure/vnet/azurerm"
  version = "2.0.0"
}
```

## Module Best Practices

1. **Keep modules focused** - One module per resource type
2. **Use descriptive variable names**
3. **Document variables with descriptions**
4. **Always output important values**
5. **Version modules** - Use Git tags or registry versions
6. **Test modules independently**
7. **Don't hardcode values** - Use variables
8. **Follow naming conventions**

---

# Debugging and Troubleshooting

## Common Terraform Errors

### 1. Syntax Errors

```
Error: Unsupported argument
on main.tf line 5, in resource "azurerm_resource_group" "example":
5:   invalid_option = true
```

**Solution:**

```
terraform validate
# Review Terraform documentation for correct syntax
```

### 2. Authentication Errors

```
Error: AccessDenied: You do not have permission to perform this action
```

**Solution:**

```
# Verify credentials
az account show
aws sts get-caller-identity

# Check permissions
# Ensure Service Principal has Contributor role
```

**3. Resource Already Exists**

```
Error: Resource group 'my-rg' already exists
```

**Solution:**

```
# Import existing resource
terraform import azurerm_resource_group.example
/subscriptions/xxx/resourceGroups/my-rg

# Or delete and recreate
```

**4. State Locking Issues**

```
Error: Error acquiring the state lock
```

**Solution:**

```
# For Azure Blob Storage
az storage blob delete --account-name mystorage --container-name tfstate --name
terraform.tfstate

# For AWS DynamoDB
aws dynamodb delete-item --table-name terraform-lock --key '{"LockID": {"S":
"terraform.tfstate"}}'
```

**5. Provider Plugin Not Found**

```
Error: Provider registry.terraform.io/hashicorp/azurerm not found
```

**Solution:**

```
terraform init
terraform init -upgrade
```

## Debugging Tools

1. **`terraform validate`** - Check syntax

```
terraform validate
```

2. **`terraform plan`** - Preview changes

```
terraform plan -out=tfplan
```

3. **`TF_LOG` for Debug Logs**

```
export TF_LOG=DEBUG
export TF_LOG_PATH=terraform.log
terraform apply
```

4. **`terraform state list` and `show`**

```
terraform state list
terraform state show azurerm_resource_group.example
```

5. **`terraform graph` for Dependency Visualization**

```
terraform graph | dot -Tpng > graph.png
```

## Troubleshooting Checklist

1. **Run `terraform validate`** - Check for syntax errors
2. **Run `terraform plan`** - See what will change
3. **Check authentication** - Verify credentials
4. **Review error message** - Read carefully
5. **Check documentation** - Verify resource syntax
6. **Enable debug logging** - `TF_LOG=DEBUG`
7. **Check state file** - `terraform state list`
8. **Verify permissions** - Ensure proper IAM roles

---

# Best Practices

## Code Organization

```
terraform-project/
├── main.tf              # Core resources
├── variables.tf         # Input variables
├── outputs.tf          # Output values
├── providers.tf        # Provider configuration
├── terraform.tfvars    # Default variable values
├── envs/               # Environment-specific values
│   ├── dev.tfvars
│   ├── staging.tfvars
│   └── prod.tfvars
├── modules/            # Custom modules
│   ├── vnet/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── vm/
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
└── .gitignore
```

## .gitignore Example

```
# Terraform
terraform.tfstate
terraform.tfstate.backup
.terraform/
.terraform.lock.hcl
*.tfvars
!*.tfvars.example
override.tf
override.tf.json
terraform.rc
```

## Naming Conventions

**Resources:**

```
resource "azurerm_resource_group" "web_app_rg" {
  name = "web-app-rg"
}
```

**Variables:**

```
variable "resource_group_name" {}
variable "vm_instance_count" {}
variable "is_production" {}
```

**Outputs:**

```
output "vnet_id" {}
output "vm_private_ips" {}
```

## Security Best Practices

1. **Never hardcode secrets** - Use variables with `sensitive = true`
2. **Use remote state with encryption**
3. **Restrict state file access** - Use RBAC
4. **Rotate credentials regularly**
5. **Use least privilege** - Minimal IAM roles
6. **Enable audit logging**
7. **Review changes with `terraform plan` before applying**
8. **Use `terraform validate` in CI/CD pipelines**

## Testing and CI/CD

1. **Run `terraform fmt`** - Format code
2. **Run `terraform validate`** - Validate syntax
3. **Run `terraform plan`** - Preview changes
4. **Use `terraform apply -auto-approve` only in CI/CD**
5. **Test in dev environment first**

## Cost Optimization

1. **Use appropriate instance sizes** - `Standard_B1s` for dev, larger for prod
2. **Delete resources when not needed** - Use `terraform destroy`
3. **Use spot instances for non-critical workloads**
4. **Implement auto-shutdown for dev environments**
5. **Monitor costs with tags**

## Complete Example: Azure VM with Module

### Project Structure

```
azure-vm-project/
├── main.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars
├── envs/
│   ├── dev.tfvars
│   └── prod.tfvars
└── modules/
    └── vnet/
        ├── main.tf
        ├── variables.tf
        └── outputs.tf
```

`main.tf`

```
terraform {
  required_version = ">= 1.0"

  backend "azurerm" {
    resource_group_name  = "terraform-state-rg"
    storage_account_name = "terraformstate123"
    container_name       = "tfstate"
    key                  = "azure-vm.tfstate"
  }
}

provider "azurerm" {
  features {}
}

# Resource Group
resource "azurerm_resource_group" "main" {
  name     = "${var.project_name}-${var.environment}-rg"
  location = var.location
  tags     = local.tags
}

# VNet Module
module "vnet" {
  source = "./modules/vnet"

  vnet_name           = "${var.project_name}-${var.environment}-vnet"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  address_space       = var.address_space
  subnet_name         = "${var.project_name}-${var.environment}-subnet"
  subnet_prefixes     = var.subnet_prefixes
}

# Public IP
resource "azurerm_public_ip" "main" {
  count = var.environment == "production" ? 0 : 1

  name                = "${var.project_name}-${var.environment}-pip"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method   = "Static"
```

```
  tags                = local.tags
}

# Network Interface
resource "azurerm_network_interface" "main" {
  name                = "${var.project_name}-${var.environment}-nic"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                = local.tags

  ip_configuration {
    name                          = "internal"
    subnet_id                     = module.vnet.subnet_id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = var.environment == "production" ? null :
azurerm_public_ip.main[0].id
  }
}

# Virtual Machine
resource "azurerm_virtual_machine" "main" {
  name                  = "${var.project_name}-${var.environment}-vm"
  location              = azurerm_resource_group.main.location
  resource_group_name   = azurerm_resource_group.main.name
  network_interface_ids = [azurerm_network_interface.main.id]
  vm_size               = var.vm_size
  tags                  = local.tags

  storage_image_reference {
    publisher = var.image_publisher
    offer     = var.image_offer
    sku       = var.image_sku
    version   = "latest"
  }

  storage_os_disk {
    name              = "${var.project_name}-${var.environment}-osdisk"
    caching           = "ReadWrite"
    create_option     = "FromImage"
    managed_disk_type = "Standard_LRS"
  }

  os_profile {
```

```
    computer_name  = "${var.project_name}-${var.environment}-vm"
    admin_username = var.admin_username
    admin_password = var.admin_password
  }

  os_profile_linux_config {
    disable_password_authentication = false
  }
}

# Local values
locals {
  tags = {
    Environment = var.environment
    Project     = var.project_name
    ManagedBy   = "Terraform"
    DeployedBy  = var.deployed_by
  }
}
```

**`variables.tf`**

```
variable "project_name" {
  description = "Name of the project"
  type        = string
  default     = "azure-vm"
}

variable "environment" {
  description = "Deployment environment (dev, staging, production)"
  type        = string
  validation {
    condition     = contains(["dev", "staging", "production"], var.environment)
    error_message = "Environment must be dev, staging, or production."
  }
}

variable "location" {
  description = "Azure region"
  type        = string
  default     = "East US"
}

variable "address_space" {
  description = "Address space for VNet"
  type        = list(string)
  default     = ["10.0.0.0/16"]
}

variable "subnet_prefixes" {
  description = "Subnet address prefixes"
  type        = list(string)
  default     = ["10.0.1.0/24"]
}

variable "vm_size" {
  description = "Size of the virtual machine"
  type        = string
  default     = "Standard_B1s"
}

variable "admin_username" {
  description = "Admin username for the VM"
  type        = string
```

```
  default     = "azureuser"
}

variable "admin_password" {
  description = "Admin password for the VM"
  type        = string
  sensitive   = true
}

variable "image_publisher" {
  description = "Image publisher"
  type        = string
  default     = "Canonical"
}

variable "image_offer" {
  description = "Image offer"
  type        = string
  default     = "UbuntuServer"
}

variable "image_sku" {
  description = "Image SKU"
  type        = string
  default     = "18.04-LTS"
}

variable "deployed_by" {
  description = "Who deployed this infrastructure"
  type        = string
  default     = "Terraform"
}
```

**`outputs.tf`**

```
output "resource_group_name" {
  description = "Name of the resource group"
  value       = azurerm_resource_group.main.name
}

output "vnet_name" {
  description = "Name of the virtual network"
  value       = module.vnet.vnet_name
}

output "subnet_id" {
  description = "ID of the subnet"
  value       = module.vnet.subnet_id
}

output "vm_name" {
  description = "Name of the virtual machine"
  value       = azurerm_virtual_machine.main.name
}

output "vm_private_ip" {
  description = "Private IP address of the VM"
  value       = azurerm_network_interface.main.private_ip_address
}

output "vm_public_ip" {
  description = "Public IP address of the VM"
  value       = var.environment == "production" ? null :
azurerm_public_ip.main[0].ip_address
}
```

## `envs/dev.tfvars`

```
environment = "dev"
vm_size     = "Standard_B1s"
admin_password = "DevPass123!"
```

## `envs/prod.tfvars`

```
environment = "production"
vm_size     = "Standard_D4s_v3"
admin_password = "ProdPass456!"
```

### Deployment Commands

```
# Initialize
terraform init

# Format code
terraform fmt

# Validate
terraform validate

# Plan for dev
terraform plan -var-file="envs/dev.tfvars" -out=dev.tfplan

# Apply dev
terraform apply dev.tfplan

# Plan for prod
terraform plan -var-file="envs/prod.tfvars" -out=prod.tfplan

# Apply prod
terraform apply prod.tfplan

# Destroy
terraform destroy -var-file="envs/dev.tfvars"
```

---

## Summary

This guide covered the complete Terraform workflow from installation to advanced modules. Key takeaways:

1. **Infrastructure as Code** eliminates manual errors and ensures consistency
2. **Terraform** is cloud-agnostic with declarative HCL syntax
3. **State file** tracks infrastructure and must be secured
4. **Variables** make configurations reusable across environments
5. **Modules** enable code reuse and team collaboration

6. **Functions and conditionals** add dynamic capabilities
7. **Debugging tools** help troubleshoot issues effectively

Remember: Always refer to official Terraform documentation, use proper naming conventions, follow security best practices, and test changes in non-production environments first.